Topic: Game recommendation bot

Problem definition

In the vast and ever-expanding universe of video games, players often find themselves overwhelmed by the sheer volume of choices available, leading to decision paralysis. A game recommendation bot serves as a navigational beacon, guiding gamers through this sea of options by analyzing their preferences, playing habits, and the types of games they enjoy. Such a bot addresses the issue of discoverability, ensuring that players are matched with games that resonate with their tastes and potentially introducing them to hidden gems they might otherwise overlook. Moreover, it streamlines the selection process, saving time and enhancing the overall gaming experience by providing personalized, curated suggestions that align with individual player profiles.

Problem solution

The chat bot is implemented in Python using the Telebot library, assists players in navigating the vast gaming universe. It addresses decision overload by recommending similar games based on tags and metacritic ratings on a given game. Personalization is achieved through commands like /Tags and /Tag, tailoring suggestions to individual preferences. The bot enhances discoverability by highlighting underrated games from specific years using the /Year command. Efficiency is ensured by recommending games based on playtime with commands like /Playtime_mainstory, /Playtime_extracontent, and /Playtime_total. Lastly, can find games through commands like /Longest_games, /Shortest_games, and /Top_games, showing games based on time of spend time by players.

Inputs & Outputs to and from Chat Bot:

Inputs: All inputs are text-based. The input can have numbers(years, hours), game names, tags.

Outputs: The bot output will be in the form of text: Recommendations, top games in specified category.

Sample use-case:

For game recommendation

Input: /Game Resident Evil 5

Output:

Game result:

name year metacritic_rating

Resident Evil Revelations 2013.0 77.0

Resident Evil 6 2013.0 69.0

Obscure 2014.0 63.0

Obscure II (Obscure: The Aftermath) 2014.0 59.0

Resident Evil: Operation Raccoon City 2012.0 48.0

Results accuracy: When searching for similar games to “Resident Evil 5” in browser the results contain the same games like in the result of the agent. Source: https://steampeek.hu/?appid=21690

Flow of System:

The first input that user will send is “/start”, the output will be the list of all commands and description. Depending on the command bot will except different values: for /Game command, bot will except name of the game; for /Playtime_total command, bot will except number of hours; etc.

PEAS Table:

Agent Type	Performance Measure	Environment	Actuators	Sensors
Game Recommendation Chat Bot	Fast, Games Recommendation Accuracy.	User who want to find games based to user’s preferences.	Screen display(user inputs, recommendations)	Keyboard entry and mouse/screen click/touch – To get user input

Environment Characteristics Table for Game Recommendation Chat Bot:

Fully Observable	**Partially Observable**
Yes, the chatbot's state is determined by the current message it receives, which is fully observable to the bot.	No, the bot doesn't need to remember past messages to respond to the current one, as each command is self-contained and doesn't depend on previous commands.
Deterministic	**Stochastic**
Yes, the chatbot's next state (the response it generates) is determined by its current state (the message it receives) and the action it takes (processing the message and generating a response).	No, there is no randomness involved in the chatbot's responses, so it is deterministic, not stochastic.
Episodic	**Sequential**
Yes, each interaction with the user (each command and response) can be considered an episode.	No, agent doesn’t have memory of past states.
Static	**Dynamic**
No, because it interacts with users in real-time.	Yes, the chatbot is interacting with users in real-time, and the state of the environment (the messages from the users) can change while the chatbot is processing a message
Discrete	**Continuous**
Yes, the chatbot responds to specific commands from the user, such as '/Game', '/Tags', '/Year', etc. Each command triggers a specific response from the bot.	No, chatbot responds to specific commands from the user and each command triggers a specific response from the bot.
Single Agent	**Multiple Agent**
Yes, the chatbot is the only agent interacting with the environment (receiving user messages and responding to them). It doesn't interact or coordinate with other agents.	No, only one bot interacts with the user.

Platform used: VS Code

Visual Studio Code (VS Code) is a free, open-source, lightweight, and powerful source code editor developed by Microsoft. It's available for Windows, macOS, and Linux. It supports a wide range of programming languages out of the box and additional languages, themes, and functionalities can be added through extensions.

Advantages of VS Code

- Syntax highlighting and smart completions with IntelliSense, which provides smart completions based on variable types, function definitions, and imported modules.
- Debugging support with breakpoints, call stacks, and an interactive console.
- Built-in Git support for version control.
- Extensibility to add new languages, themes, and features through plugins.
- Integrated terminal with support for multiple instances.
- Support for Jupyter notebooks through the Python extension.
- A rich API for developers to create their own extensions.

VS Code is highly customizable, allowing users to change the editor's theme, keyboard shortcuts, and preferences. It also has a large and active community, which contributes to a vast selection of extensions to enhance its functionality.

Working of a chatbot on VS Code

This script is essentially a chatbot for game recommendations. Here's a step-by-step breakdown:

1. The bot starts by importing necessary libraries and functions. It also loads data from a CSV file and a pickle file, which contain information about games.

2. The bot is set up to respond to various commands. When a user sends a command from Telegram, the bot processes it and responds accordingly.

3. If the user asks for a game recommendation (/Game command), the bot takes the name of the game, processes it, and uses a function to find similar games. It then sends the results back to the user in Telegram.

4. If the user asks for all game tags (/Tags command), the bot retrieves all the tags from the data, sorts them by popularity, and sends them to the user in Telegram.

5. If the user asks for top games based on a tag (/Tag command), the bot takes the tag, processes it, and uses a function to find the top games for that tag. It then sends the results back to the user in Telegram.

6. The bot also responds to other commands like /Year, /Playtime_mainstory, /Playtime_extracontent, /Playtime_total, /Longest_games, /Shortest_games, and /Top_games. For each of these commands, the bot processes the user's input, uses a function to find the relevant games, and sends the results back to the user in Telegram.

7. Finally, the bot starts polling, which means it continuously checks for new messages from users and responds to them as described above.

Screenshots:

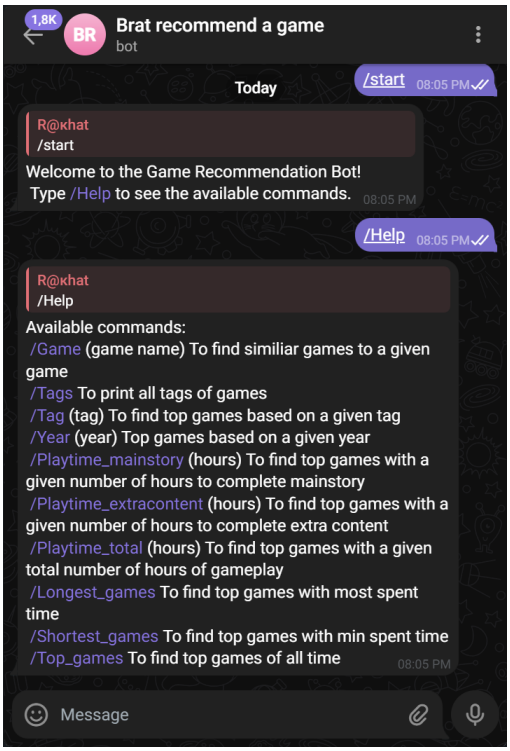


Figure 1 Start

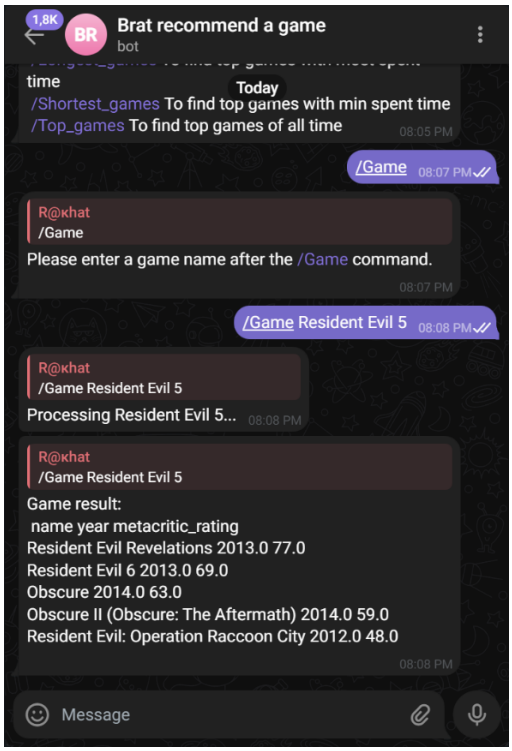


Figure 2 /Game command

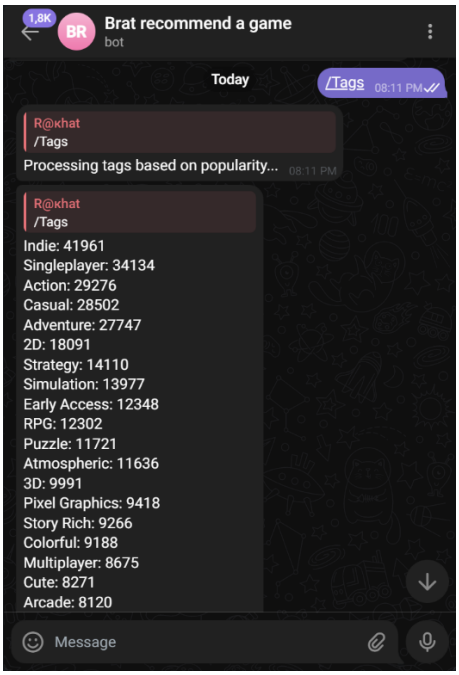


Figure 3 /Tag Command

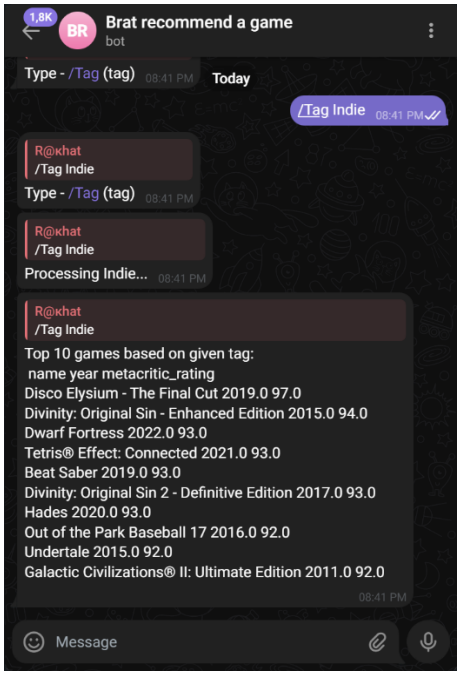


Figure 4 /Tags Command

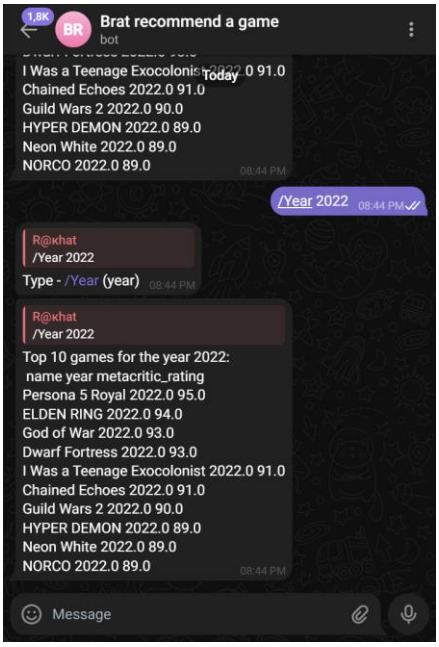


Figure 5 /Playtime_mainstory Command

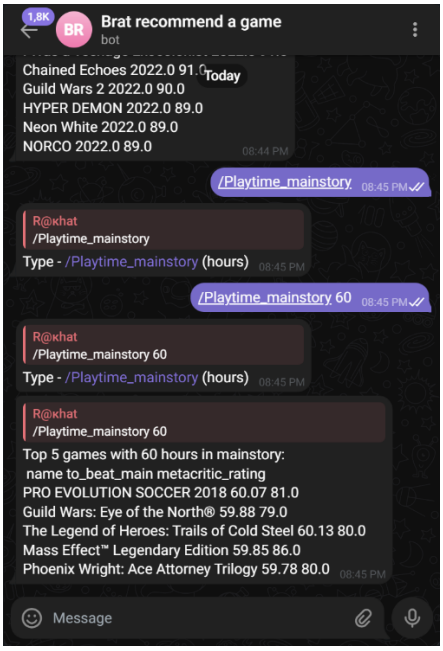


Figure 6 /Year Command

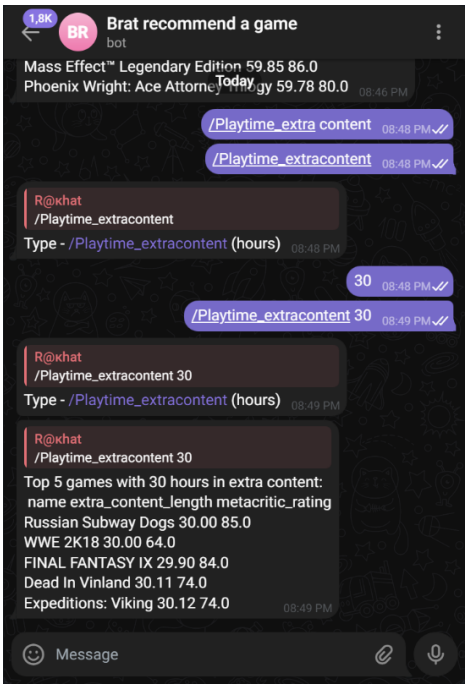


Figure 7 /Playtime_total Command

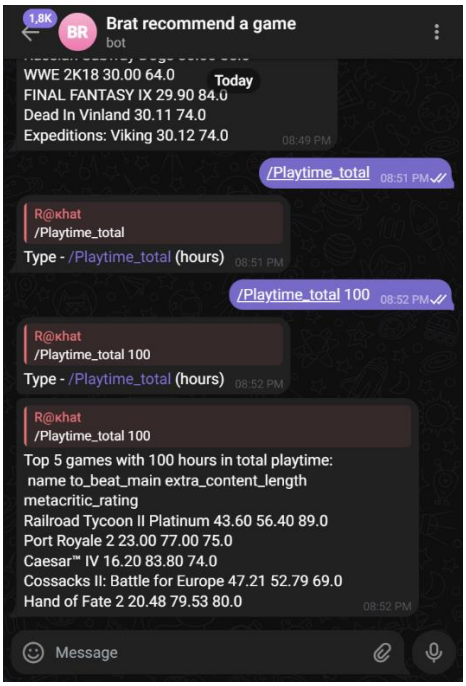


Figure 8 /Playtime_extracontent Command

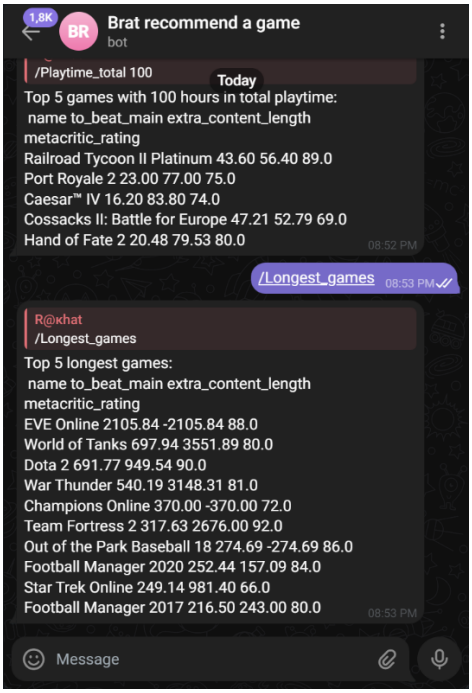


Figure 9 /Longest_games

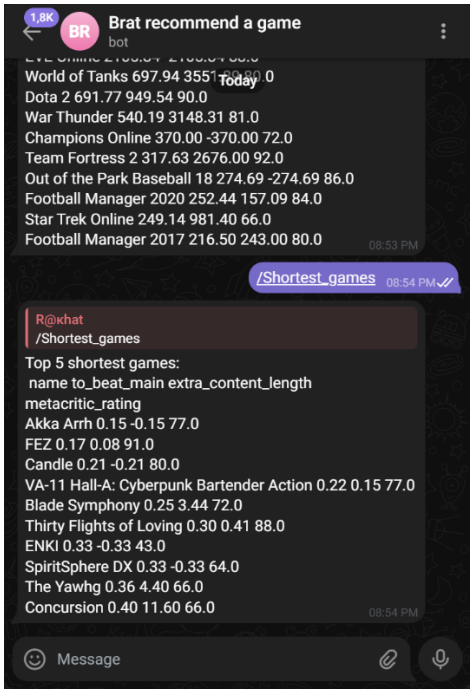


Figure 10 /Shortest_games Command

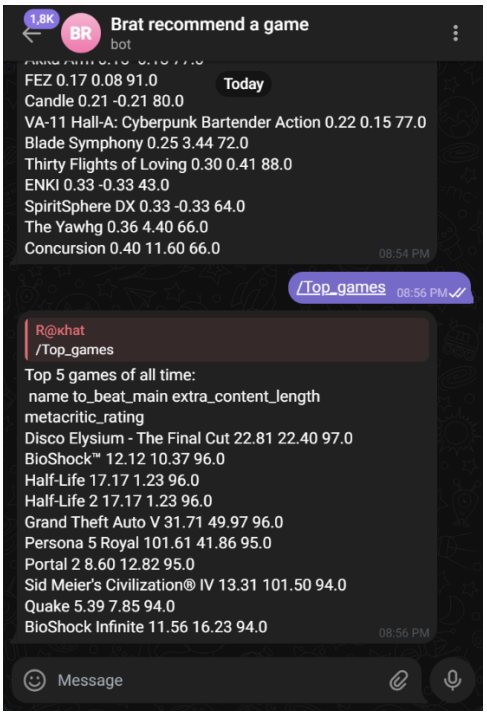


Figure 11 /Top_games Command



Figure 12 Result of: plt.scatter(df.reviewer_rating, df.tags,marker='',color='red')*

Conclusion:

In summary, the development of a game recommendation bot using Python and the Telebot library within Visual Studio Code represents a significant step forward in assisting gamers overwhelmed by the vast selection of video games available. This bot efficiently addresses decision paralysis by providing personalized recommendations based on individual preferences, playing habits, and game attributes. By leveraging commands like /Game, /Tags, /Year, and others, users can quickly access tailored suggestions and discover new games aligned with their interests, ultimately saving time and enriching the gaming experience.

By Rakhat Bektas